

運用 TensorFlow Agents 和 Flutter 打造桌遊

來源：<https://codelabs.developers.google.com/tfagents-flutter?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將建構採用機器學習技術的簡易桌遊。您將透過 TensorFlow Agents 訓練強化學習模型，並以 TensorFlow Serving 做為後端來部署這個模型。此外，您也將建構跨平台 Flutter 應用程式做為遊戲前端。

目錄

1. 事前準備
2. The Plane Strike Game
3. 設定 Flutter 開發環境
4. 做好準備
5. 下載專案的依附元件
6. 步驟 0：執行範例應用程式
7. 步驟 1：建立 TensorFlow Agents Python 環境
8. 步驟 2：使用 TensorFlow Agents 訓練遊戲代理
9. 步驟 3：使用 TensorFlow Serving 部署訓練好的模型
10. 步驟 4：為 Android 和 iOS 建立 Flutter 應用程式
11. 步驟 5：為電腦平台啟用 Flutter 應用程式
12. 步驟 6：為網頁平台啟用 Flutter 應用程式
13. 恭喜

1. 事前準備

[AlphaGo](#) 和 [AlphaStar](#) 的重大突破，展現了運用機器學習技術打造超人級遊戲代理程式的潛力。建構小型機器學習遊戲是很有趣的練習，可幫助您掌握建立強大遊戲代理程式所需的技能。

在本程式碼研究室中，您將瞭解如何使用下列項目建構桌遊：

- TensorFlow Agent，可透過強化學習訓練遊戲虛擬對手
- 使用 TensorFlow Serving 提供模型
- 使用 Flutter 建立跨平台桌遊應用程式

必要條件

- 具備使用 Dart 進行 Flutter 開發的基本知識
- 具備 TensorFlow 機器學習基本知識，例如訓練與部署的差異
- Python、終端機和 Docker 的基本知識

課程內容

- 如何使用 TensorFlow Agents 訓練非玩家角色 (NPC) 代理程式
- 如何使用 TensorFlow Serving 提供訓練好的模型
- 如何建構跨平台的 Flutter 桌遊

軟硬體需求

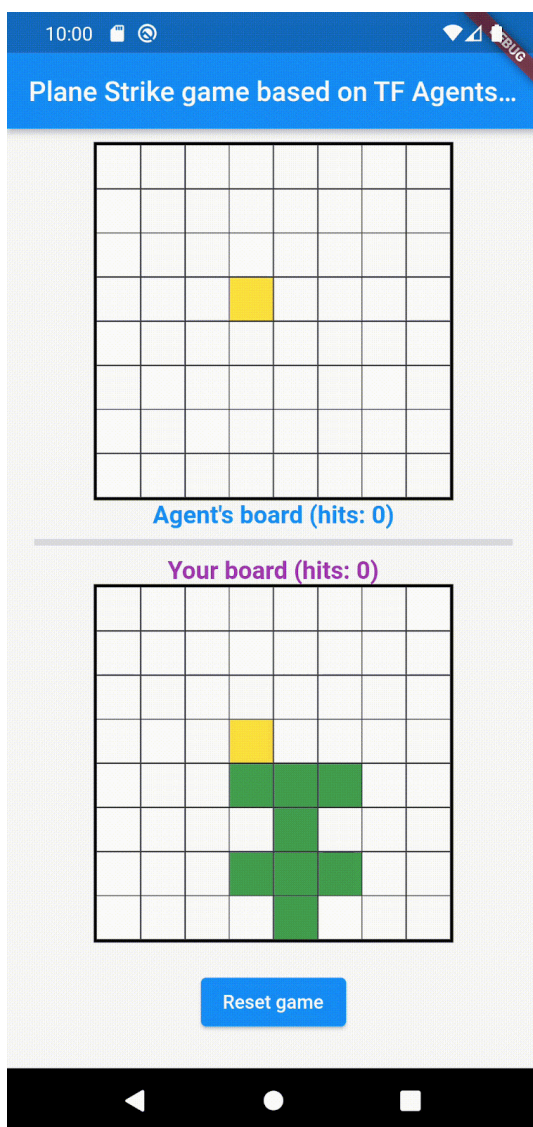
- [Flutter SDK](#)
 - 為 Flutter 設定 [Android](#) 和 [iOS](#)
 - Flutter 的[電腦](#)設定
 - Flutter 的[網頁](#)設定
 - 設定 [Visual Studio Code \(VS Code\)](#)，以便使用 [Flutter](#) 和 [Dart](#)
 - [Docker](#)
 - [Bash](#)
 - Python 3.7 以上版本
-

2. The Plane Strike Game

在本程式碼研究室中，您將建構名為「Plane Strike」的遊戲，這是一款類似於「戰艦」的雙人桌遊。規則非常簡單：

- 人類玩家會與透過機器學習訓練的 NPC 代理程式對戰。人類玩家可以輕觸代理程式棋盤中的任一儲存格，開始遊戲。
- 遊戲開始時，人類玩家和代理程式的棋盤上都會有「飛機」物件 (8 個綠色儲存格，形成「飛機」形狀，如下方動畫中人類玩家的棋盤所示)。這些「飛機」是隨機放置，只有棋盤擁有者看得到，對手則看不到。
- 人類玩家和代理程式輪流攻擊對方棋盤上的其中一個儲存格。人類玩家可以輕觸代理程式棋盤中的任何儲存格，而代理程式會根據機器學習模型的預測結果自動做出選擇。如果嘗試的儲存格是「飛機」儲存格 (「擊中」)，則會變成紅色；否則會變成黃色 (「未擊中」)。
- 先達到 8 個紅色儲存格者獲勝，接著遊戲會重新開始，並使用新的棋盤。

以下是遊戲的試玩影片：



3. 設定 Flutter 開發環境

如要進行 Flutter 開發，您需要兩項軟體才能完成本程式碼研究室：[Flutter SDK](#) 和[編輯器](#)。

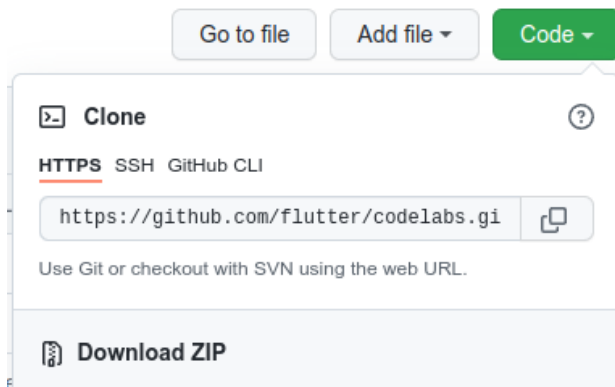
您可以使用下列任一裝置執行程式碼研究室：

- [iOS 模擬器](#) (需要安裝 Xcode 工具)。
 - [Android Emulator](#) (需在 Android Studio 中設定)。
 - 瀏覽器 (偵錯時必須使用 Chrome)。
 - 以 [Windows](#)、[Linux](#) 或 [macOS](#) 電腦版應用程式的形式。您必須在要部署的平台上進行開發。因此，如要開發 Windows 桌面應用程式，您必須在 Windows 上開發，才能存取適當的建構鏈。如需作業系統專屬需求，請參閱 docs.flutter.dev/desktop。
-

4. 做好準備

如要下載本程式碼研究室的程式碼，請按照下列步驟操作：

1. 前往本程式碼研究室的 [GitHub 存放區](#)。
2. 依序點選「Code」>「Download zip」，下載這個程式碼研究室的所有程式碼。



3. 將下載的 ZIP 檔案解壓縮，解壓縮後會產生 `codelabs-main` 根資料夾，內含所有需要的資源。

在本程式碼研究室中，您只需要存放區中 `tfagents-flutter/` 子目錄的檔案，其中包含多個資料夾：

- `step0` 至 `step6` 資料夾包含範例程式碼，您將在本程式碼研究室的每個步驟中，以這些程式碼為基礎進行建構。
 - `finished` 資料夾包含完成的程式碼，適用於完成的範例應用程式。
 - 每個資料夾都包含 `backbend` 子資料夾 (內含後端程式碼) 和 `frontend` 子資料夾 (內含 Flutter 前端程式碼)
-

步驟 5 · 約 3 分鐘

5. 下載專案的依附元件

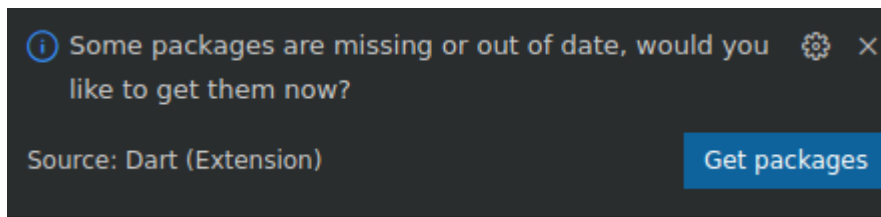
後端

開啟終端機並進入 `tfagents-flutter` 子資料夾。執行以下指令：

```
pip install -r requirements.txt
```

前端

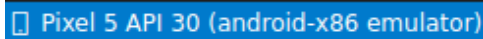
1. 在 VS Code 中，依序點選「File」>「Open folder」，然後選取先前下載的原始碼中的 `step0` 資料夾。
2. 開啟 `step0/frontend/lib/main.dart` 檔案如果看到 VS Code 對話方塊，提示您下載範例應用程式的必要套件，請按一下「Get packages」(取得套件)。
3. 如果沒有看到這個對話方塊，請開啟終端機，然後在 `step0/frontend` 資料夾中執行 `flutter pub get` 指令。



6. 步驟 0：執行範例應用程式

1. 在 VS Code 中開啟 `step0/frontend/lib/main.dart` 檔案，確認 Android Emulator 或 iOS 模擬器已正確設定，並顯示在狀態列中。

舉例來說，在 Android 模擬器中使用 Pixel 5 時，畫面會顯示以下內容：



以下是使用 iOS 模擬器搭配 iPhone 13 時的畫面：



2. 按一下「Start debugging」



。

注意：本程式碼研究室假設您在本機電腦上部署模型。如果要在其他遠端機器上部署模型，請將程式碼中的 `10.0.2.2` 或 `127.0.0.1` IP 位址變更為執行 TensorFlow Serving 的遠端機器 IP 位址。

執行並探索應用程式

應用程式應會在 Android 模擬器或 iOS 模擬器上啟動。使用者介面相當簡單明瞭。畫面會顯示 2 個遊戲棋盤。人類玩家可以輕觸頂端代理程式棋盤中的任何儲存格，做為攻擊位置。您將訓練智慧代理程式，根據人類玩家的棋盤自動預測攻擊位置。

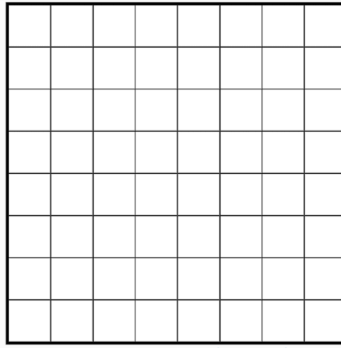
在幕後，Flutter 應用程式會將人類玩家目前的棋盤傳送至後端，後端會執行強化學習模型，並傳回預測的下一個落子位置。前端收到回應後，就會在 UI 中顯示結果。

5:51



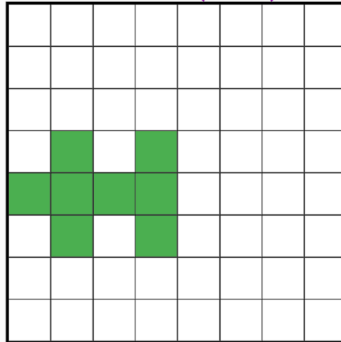
bug

Plane Strike game based on TF Agents...

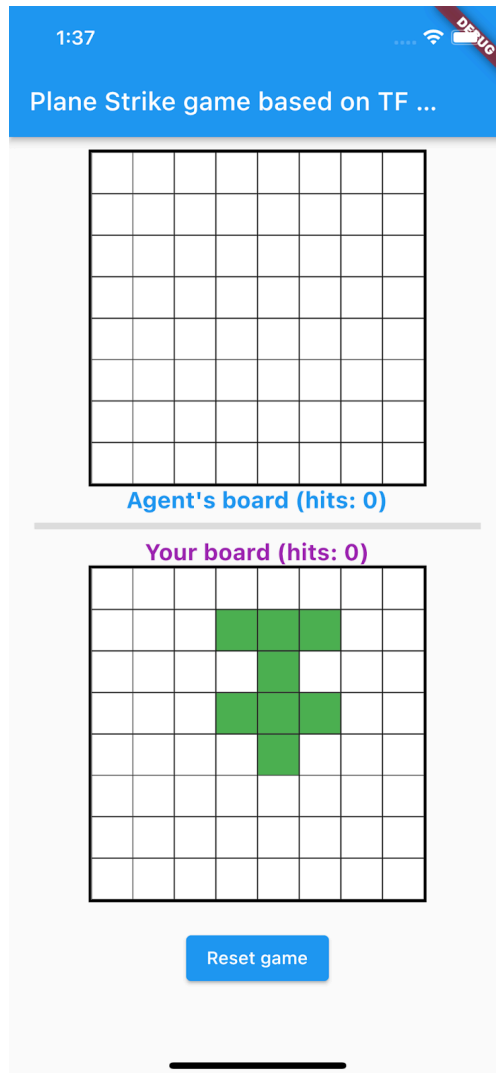


Agent's board (hits: 0)

Your board (hits: 0)



Reset game

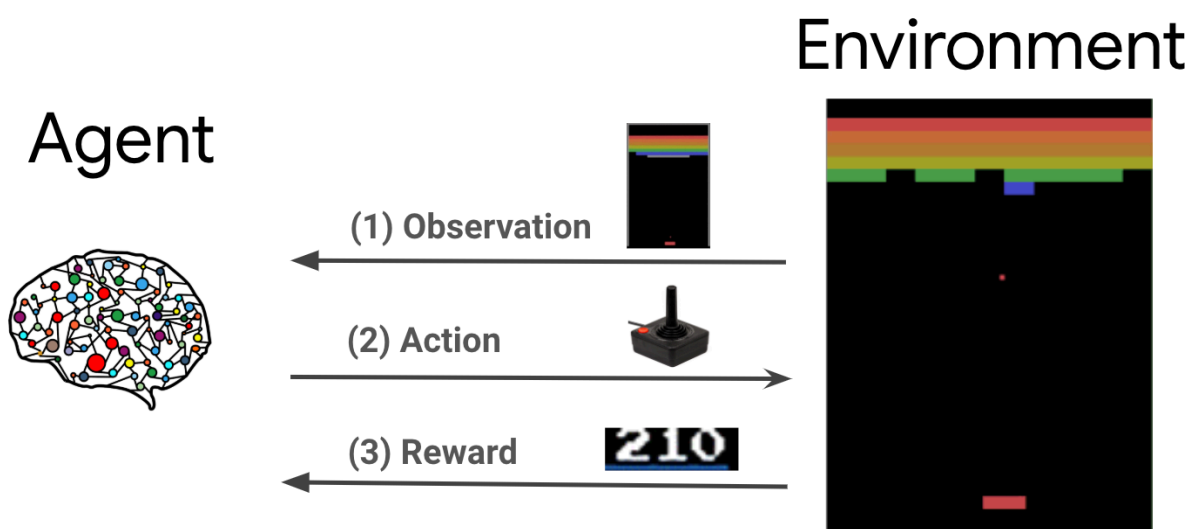


如果您現在點選代理程式面板中的任何儲存格，系統不會有任何反應，因為應用程式還無法與後端通訊。

7. 步驟 1：建立 TensorFlow Agents Python 環境

本程式碼研究室的主要目標是設計代理程式，透過與環境互動來學習。雖然「飛機攻擊」遊戲相對簡單，可以為 NPC 代理手動製作規則，但您會使用強化學習訓練代理，藉此學習相關技能，日後就能輕鬆為其他遊戲建構代理。

在標準強化學習 (RL) 設定中，代理程式會在每個時間步收到觀察結果，並選擇動作。系統會將動作套用至環境，環境則會傳回回饋和新的觀察結果。代理會訓練政策，選擇可將獎勵總和 (又稱回報) 提高到最高的動作。透過反覆玩遊戲，代理程式可以學習模式並磨練技能，最終精通遊戲。如要將「飛機大戰」遊戲制定為 RL 問題，請將棋盤狀態視為觀察結果、攻擊位置視為動作，以及擊中/未擊中信號視為回饋。



注意：如要深入瞭解強化學習，請參閱 [DeepMind](#) 和 [UCL](#) 於 2021 年舉辦的強化學習講座系列。

如要訓練 NPC 代理程式，請使用 TensorFlow Agents。這款強化學習程式庫適用於 TensorFlow，不僅可靠、可擴充，而且簡單易用。

TF-Agents 隨附大量程式碼研究室、範例和詳盡說明文件，可協助您入門，非常適合用於強化學習。您可以使用 TF Agents 解決實際且複雜的 RL 問題，並快速開發新的 RL 演算法。您可以輕鬆切換不同的代理程式和演算法，進行實驗。此外，這個版本也經過充分測試，且容易設定。

[OpenAI Gym](#) (例如 Atari 遊戲)、[Mujoco](#) 等平台實作了許多預先建構的遊戲環境，TF-Agents 可輕鬆運用這些環境。但由於 Plane Strike 遊戲是完全自訂的遊戲，您必須先從頭實作新的環境。

注意：如要進一步瞭解 TensorFlow Agents 環境，請參閱這篇 [TensorFlow Agents 教學課程](#)。

如要實作 TF-Agents Python 環境，您需要實作下列方法：

```

class YourGameEnv(py_environment.PyEnvironment):

    def __init__(self):
        """Initialize environment."""

    def action_spec(self):
        """Return action_spec."""

    def observation_spec(self):
        """Return observation_spec."""

    def _reset(self):
        """Return initial_time_step."""

    def _step(self, action):
        """Apply action and return new time_step."""

```

最重要的函式是 `_step()` 函式，該函式會接收動作並傳回新的 `time_step` 物件。以「飛機轟炸」遊戲為例，您有一個遊戲棋盤；當新的轟炸位置出現時，環境會根據遊戲棋盤的狀況，判斷出：

- 遊戲棋盤接下來應呈現的樣貌 (如果單元格的顏色應變更為紅色或黃色，則會顯示隱藏的飛機位置)
- 玩家應獲得該位置的獎勵 (命中獎勵或失誤懲罰)？
- 遊戲是否應終止 (是否有人獲勝？)
- 將下列程式碼加入 `_planestrike_py_environment.py` 檔案的 `_step()` 函式中：

```

if self._hit_count == self._plane_size:
    self._episode_ended = True
    return self.reset()

if self._strike_count + 1 == self._max_steps:
    self.reset()
    return ts.termination(
        np.array(self._visible_board, dtype=np.float32), UNFINISHED_GAME_REWARD
    )

self._strike_count += 1
action_x = action // self._board_size
action_y = action % self._board_size
# Hit
if self._hidden_board[action_x][action_y] == HIDDEN_BOARD_CELL_OCCUPIED:
    # Non-repeat move
    if self._visible_board[action_x][action_y] == VISIBLE_BOARD_CELL_UNTRIED:
        self._hit_count += 1
        self._visible_board[action_x][action_y] = VISIBLE_BOARD_CELL_HIT
        # Successful strike
        if self._hit_count == self._plane_size:
            # Game finished
            self._episode_ended = True
            return ts.termination(
                np.array(self._visible_board, dtype=np.float32),
                FINISHED_GAME_REWARD,
            )
        else:
            self._episode_ended = False
            return ts.transition(
                np.array(self._visible_board, dtype=np.float32),
                HIT_REWARD,
                self._discount,
            )
    # Repeat strike
    else:
        self._episode_ended = False
        return ts.transition(
            np.array(self._visible_board, dtype=np.float32),
            REPEAT_STRIKE_REWARD,
            self._discount,
        )
# Miss
else:
    # Unsuccessful strike
    self._episode_ended = False
    self._visible_board[action_x][action_y] = VISIBLE_BOARD_CELL_MISS
    return ts.transition(

```

```
np.array(self._visible_board, dtype=np.float32),  
MISS_REWARD,  
self._discount,
```

8. 步驟 2：使用 TensorFlow Agents 訓練遊戲代理

有了 TF-Agents 環境，您就可以訓練遊戲代理。在本程式碼研究室中，您會使用 REINFORCE 代理程式。REINFORCE 是 RL 中的策略梯度演算法，基本概念是根據遊戲過程中收集到的獎勵信號調整政策類神經網路參數，讓政策網路在日後的遊戲中盡量爭取回報。

- 首先，您需要例項化訓練和評估環境。將這段程式碼新增至 `step2/backend/training.py` 檔案中的 `train_agent()` 函式：

```
train_py_env = planestrike_py_environment.PlaneStrikePyEnvironment(
    board_size=BOARD_SIZE, discount=DISCOUNT, max_steps=BOARD_SIZE**2
)
eval_py_env = planestrike_py_environment.PlaneStrikePyEnvironment(
    board_size=BOARD_SIZE, discount=DISCOUNT, max_steps=BOARD_SIZE**2
)

train_env = tf_py_environment.TFPyEnvironment(train_py_env)
eval_env = tf_py_environment.TFPyEnvironment(eval_py_env)
```

- 接著，您需要建立要訓練的強化學習代理程式。在本程式碼研究室中，您將使用 REINFORCE 代理程式，這是以政策為基礎的代理程式。在上述程式碼下方新增下列程式碼：

```
actor_net = tf.nn.nn.Sequential(
    [
        tf.nn.layers.InnerReshape([BOARD_SIZE, BOARD_SIZE], [BOARD_SIZE**2]),
        tf.nn.layers.Dense(FC_LAYER_PARAMS, activation="relu"),
        tf.nn.layers.Dense(BOARD_SIZE**2),
        tf.nn.layers.Lambda(lambda t: tf.nn.distributions.Categorical(logits=t)),
    ],
    input_spec=train_py_env.observation_spec(),
)

optimizer = tf.nn.keras.optimizers.Adam(learning_rate=LEARNING_RATE)

train_step_counter = tf.Variable(0)

tf_agent = reinforce_agent.ReinforceAgent(
    train_env.time_step_spec(),
    train_env.action_spec(),
    actor_network=actor_net,
    optimizer=optimizer,
    normalize_returns=True,
    train_step_counter=train_step_counter,
)
```

- 最後，在訓練迴圈中訓練代理程式。在迴圈中，您首先會將幾集遊戲過程收集到緩衝區，然後使用緩衝資料訓練代理程式。將這段程式碼新增至 `step2/backend/training.py` 檔案中的 `train_agent()` 函式：

```
# Collect a few episodes using collect_policy and save to the replay buffer.
collect_episode(
    train_py_env,
    collect_policy,
    COLLECT_EPISODES_PER_ITERATION,
    replay_buffer_observer,
)

# Use data from the buffer and update the agent's network.
iterator = iter(replay_buffer.as_dataset(sample_batch_size=1))
trajectories, _ = next(iterator)
tf_agent.train(experience=trajectories)
replay_buffer.clear()
```

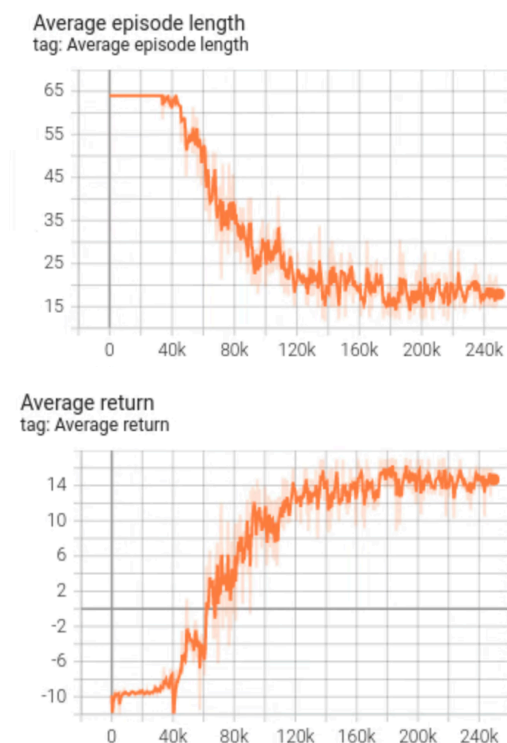
- 現在可以開始訓練。在終端機中，前往電腦上的 `step2/backend` 資料夾並執行下列指令：

```
python training.py
```

視硬體設定而定，訓練完成需要 8 到 12 小時 (由於 `step3` 提供預先訓練的模型，您不必自行完成整個訓練)。在此期間，您可以使用 [TensorBoard](#) 監控進度。開啟新的終端機，前往電腦上的 `step2/backend` 資料夾，然後執行：

```
tensorboard --logdir tf_agents_log/
```

`tf_agents_log` 是包含訓練記錄的資料夾。訓練執行範例如下所示：



您可以看到，隨著訓練進行，平均集數長度會縮短，平均報酬則會增加。直覺上，您可以瞭解，如果代理程式更聰明且能做出更準確的預測，遊戲時間就會縮短，代理程式也會獲得更多獎勵。這是合理的做法，因為代理程式希望在較少的步驟內完成遊戲，以盡量減少後續步驟中的大量獎勵折扣。

訓練完成後，訓練好的模型會匯出至 `policy_model` 資料夾。

注意：如要進一步瞭解如何使用 TensorFlow Agents 訓練 REINFORCE 代理程式，請參閱這篇 [TensorFlow Agents 教學課程](#)。

9. 步驟 3：使用 TensorFlow Serving 部署訓練好的模型

遊戲虛擬對手訓練完成後，即可使用 TensorFlow Serving 部署。

注意：為方便起見，我們在 `step3/policy_model` 資料夾中提供預先訓練的 SavedModel。

- 在終端機中，前往電腦上的 `step3/backend` 資料夾，然後使用 Docker 啟動 TensorFlow Serving：

```
docker run -t --rm -p 8501:8501 -p 8500:8500 -v  
"$(pwd)/backend/policy_model:/models/policy_model" -e MODEL_NAME=policy_model  
tensorflow/serving
```

Docker 會先自動下載 TensorFlow Serving 映像檔，這需要一分鐘。TensorFlow Serving 隨即會啟動。記錄應如下列程式碼片段所示：

```
2022-05-30 02:38:54.147771: I tensorflow_serving/model_servers/server.cc:89] Building
single TensorFlow model file config: model_name: policy_model model_base_path:
/models/policy_model
2022-05-30 02:38:54.148222: I tensorflow_serving/model_servers/server_core.cc:465]
Adding/updating models.
2022-05-30 02:38:54.148273: I tensorflow_serving/model_servers/server_core.cc:591]
(Re-)adding model: policy_model
2022-05-30 02:38:54.262684: I tensorflow_serving/core/basic_manager.cc:740]
Successfully reserved resources to load servable {name: policy_model version: 123}
2022-05-30 02:38:54.262768: I tensorflow_serving/core/loader_harness.cc:66] Approving
load for servable version {name: policy_model version: 123}
2022-05-30 02:38:54.262787: I tensorflow_serving/core/loader_harness.cc:74] Loading
servable version {name: policy_model version: 123}
2022-05-30 02:38:54.265010: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:38] Reading SavedModel
from: /models/policy_model/123
2022-05-30 02:38:54.277811: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:90] Reading meta graph
with tags { serve }
2022-05-30 02:38:54.278116: I
external/org_tensorflow/tensorflow/cc/saved_model/reader.cc:132] Reading SavedModel
debug info (if present) from: /models/policy_model/123
2022-05-30 02:38:54.280229: I
external/org_tensorflow/tensorflow/core/platform/cpu_feature_guard.cc:142] This
TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to
use the following CPU instructions in performance-critical operations: AVX2 FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler
flags.
2022-05-30 02:38:54.332352: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:206] Restoring SavedModel
bundle.
2022-05-30 02:38:54.337000: I
external/org_tensorflow/tensorflow/core/platform/profile_utils/cpu_utils.cc:114] CPU
Frequency: 2193480000 Hz
2022-05-30 02:38:54.402803: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:190] Running
initialization op on SavedModel bundle at path: /models/policy_model/123
2022-05-30 02:38:54.410707: I
external/org_tensorflow/tensorflow/cc/saved_model/loader.cc:277] SavedModel load for
tags { serve }; Status: success: OK. Took 145695 microseconds.
2022-05-30 02:38:54.412726: I
tensorflow_serving/servables/tensorflow/saved_model_warmup_util.cc:59] No warmup data
file found at /models/policy_model/123/assets.extra/tf_serving_warmup_requests
2022-05-30 02:38:54.417277: I tensorflow_serving/core/loader_harness.cc:87]
Successfully loaded servable version {name: policy_model version: 123}
2022-05-30 02:38:54.419846: I tensorflow_serving/model_servers/server_core.cc:486]
Finished adding/updating models
2022-05-30 02:38:54.420066: I tensorflow_serving/model_servers/server.cc:367]
```

```
Profiler service is enabled
2022-05-30 02:38:54.428339: I tensorflow_serving/model_servers/server.cc:393] Running
gRPC ModelServer at 0.0.0.0:8500 ...
[warn] getaddrinfo: address family for nodename not supported
2022-05-30 02:38:54.431620: I tensorflow_serving/model_servers/server.cc:414]
Exporting HTTP/REST API at:localhost:8501 ...
[evhttp_server.cc : 245] NET_LOG: Entering the event loop ...
```

注意：啟動 Tensorflow Serving Docker 映像檔後，您可以前往

http://localhost:8501/v1/models/<MODEL_NAME>/metadata，檢查輸入和輸出張量的詳細資料。在本程式碼研究室中，請將 <MODEL_NAME> 替換為 `policy_model`。

您可以向端點傳送範例要求，確認端點是否正常運作：

```
curl -d '{"signature_name":"action","instances":[{"0/discount":0.0,"0/observation":
[[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0]],"0/reward":0.0,"0/step_type":0}]}' -X POST
http://localhost:8501/v1/models/policy_model:predict
```

端點會傳回預測位置 `45`，也就是棋盤中央的 (5, 5)。(如果您好奇，可以試著找出棋盤中央是第一個打擊位置的合理原因)。

```
{
  "predictions": [45]
}
```

大功告成！您已成功建構後端，可預測 NPC 代理程式的下一個打擊位置。

注意：對於這個特定的強化學習模型，輸入內容中的 `discount`、`reward` 和 `step_type` 參數並不重要，因為這些參數只會影響訓練程序。只有 `observation` 參數會影響推論結果。

10. 步驟 4：為 Android 和 iOS 建立 Flutter 應用程式

後端已準備就緒，您可以開始傳送要求，從 Flutter 應用程式擷取罷工位置預測。

- 首先，您需要定義一個類別，用來包裝要傳送的輸入內容。將這段程式碼新增至 `step4/frontend/lib/game_agent.dart` 檔案：

```
class Inputs {
  final List<double> _boardState;
  Inputs(this._boardState);

  Map<String, dynamic> toJson() {
    final Map<String, dynamic> data = <String, dynamic>{};
    data['0/discount'] = [0.0];
    data['0/observation'] = [_boardState];
    data['0/reward'] = [0.0];
    data['0/step_type'] = [0];
    return data;
  }
}
```

現在您可以將要求傳送至 TensorFlow Serving，進行預測。

- 將這段程式碼新增至 `step4/frontend/lib/game_agent.dart` 檔案中的 `predict()` 函式：

```
var flattenedBoardState = boardState.expand((i) => i).toList();
final response = await http.post(
  Uri.parse('http://$server:8501/v1/models/policy_model:predict'),
  body: jsonEncode(<String, dynamic>{
    'signature_name': 'action',
    'instances': [Inputs(flattenedBoardState)]
  })),
);

if (response.statusCode == 200) {
  var output = List<int>.from(
    jsonDecode(response.body)['predictions'] as List<dynamic>);
  return output[0];
} else {
  throw Exception('Error response');
}
```

應用程式收到後端的回應後，您會更新遊戲 UI，反映遊戲進度。

- 將這段程式碼新增至 `step4/frontend/lib/main.dart` 檔案中的 `_gridItemTapped()` 函式：

```
int agentAction =
    await _policyGradientAgent.predict(_playerVisibleBoardState);
_agentActionX = agentAction ~/ _boardSize;
_agentActionY = agentAction % _boardSize;
if (_playerHiddenBoardState[_agentActionX][_agentActionY] ==
    hiddenBoardCellOccupied) {
    // Non-repeat move
    if (_playerVisibleBoardState[_agentActionX][_agentActionY] ==
        visibleBoardCellUntried) {
        _agentHitCount++;
    }
    _playerVisibleBoardState[_agentActionX][_agentActionY] =
        visibleBoardCellHit;
} else {
    _playerVisibleBoardState[_agentActionX][_agentActionY] =
        visibleBoardCellMiss;
}
setState(() {});
```

開始執行

1. 按一下「Start debugging」圖示



，然後等待應用程式載入。

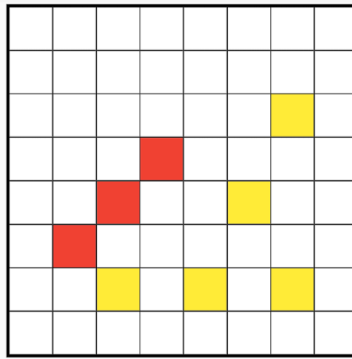
2. 輕觸特工棋盤中的任一格，即可開始遊戲。

5:51



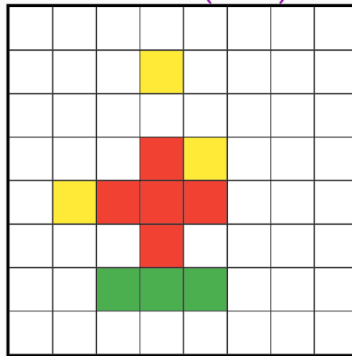
BUG

Plane Strike game based on TF Agents...



Agent's board (hits: 3)

Your board (hits: 5)



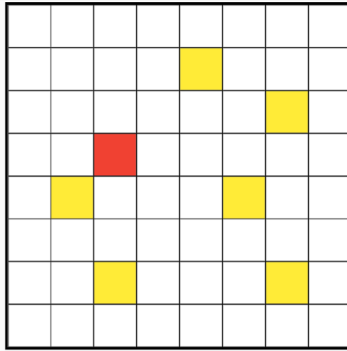
Reset game

10:39



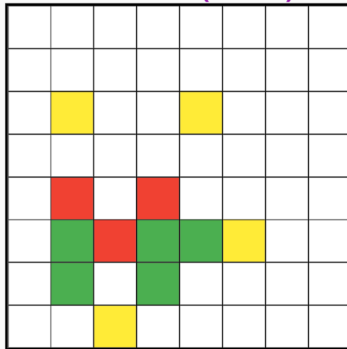
debug

Plane Strike game based on TF ...



Agent's board (hits: 1)

Your board (hits: 3)



Reset game

步驟 11 · 約 10 分鐘

11. 步驟 5：為電腦平台啟用 Flutter 應用程式

除了 Android 和 iOS，Flutter 也支援 Linux、Mac 和 Windows 等電腦平台。

Linux

1. 確認目標裝置在 VSCode 的狀態列中設為

Linux (linux-x64)

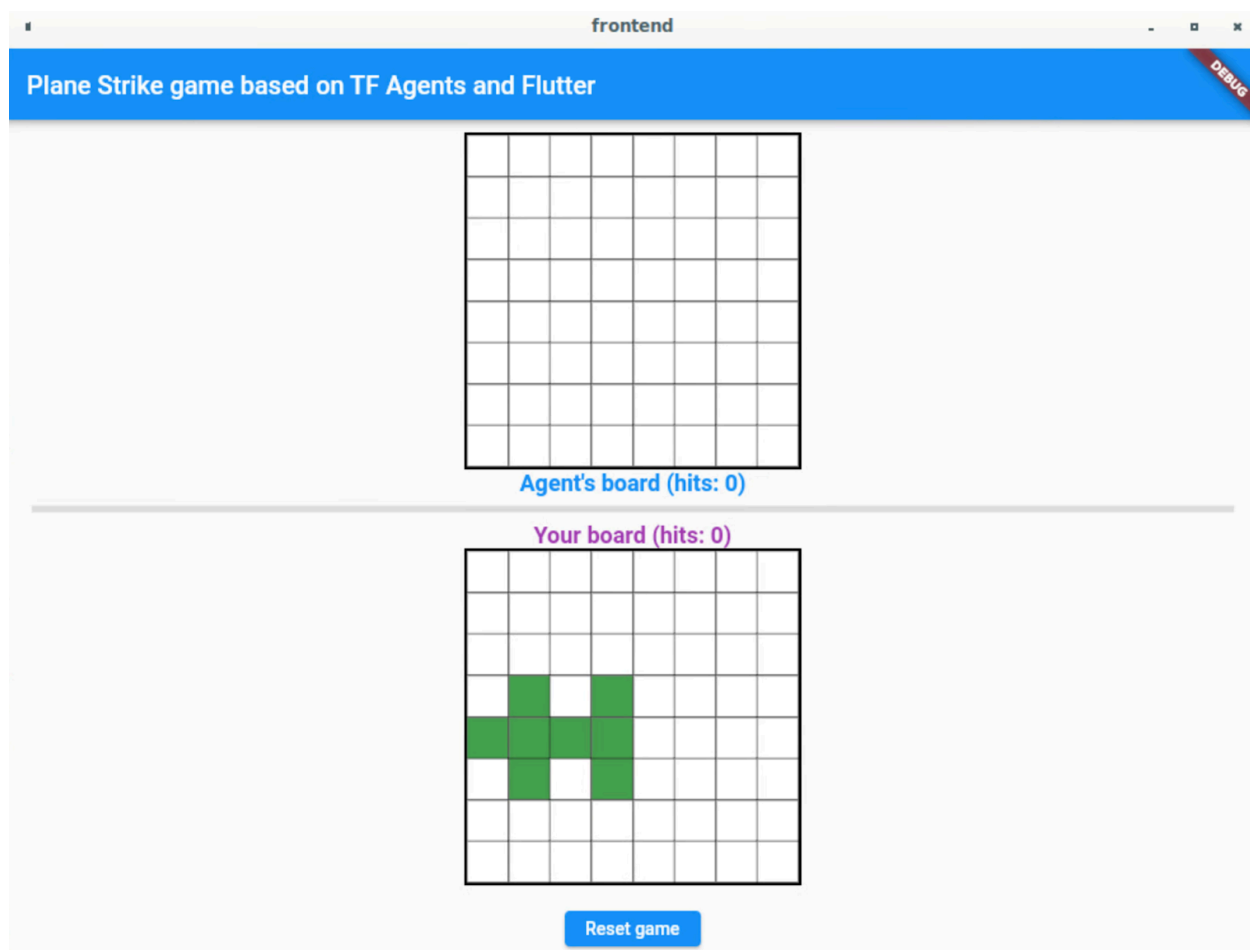
。

2. 按一下「Start debugging」圖示



，然後等待應用程式載入。

3. 按一下代理程式看板中的任一儲存格，即可開始遊戲。



Mac

1. 如果是 Mac，由於應用程式會將 HTTP 要求傳送至後端，因此您需要設定適當的權利。詳情請參閱「[Entitlements and the App Sandbox](#)」。

將這段程式碼分別新增至 `step4/frontend/macOS/Runner/DebugProfile.entitlements` 和 `step4/frontend/macOS/Runner/Release.entitlements`：

```
<key>com.apple.security.network.client</key>  
<true/>
```

2. 確認目標裝置在 VSCode 的狀態列中設為

macOS (darwin)

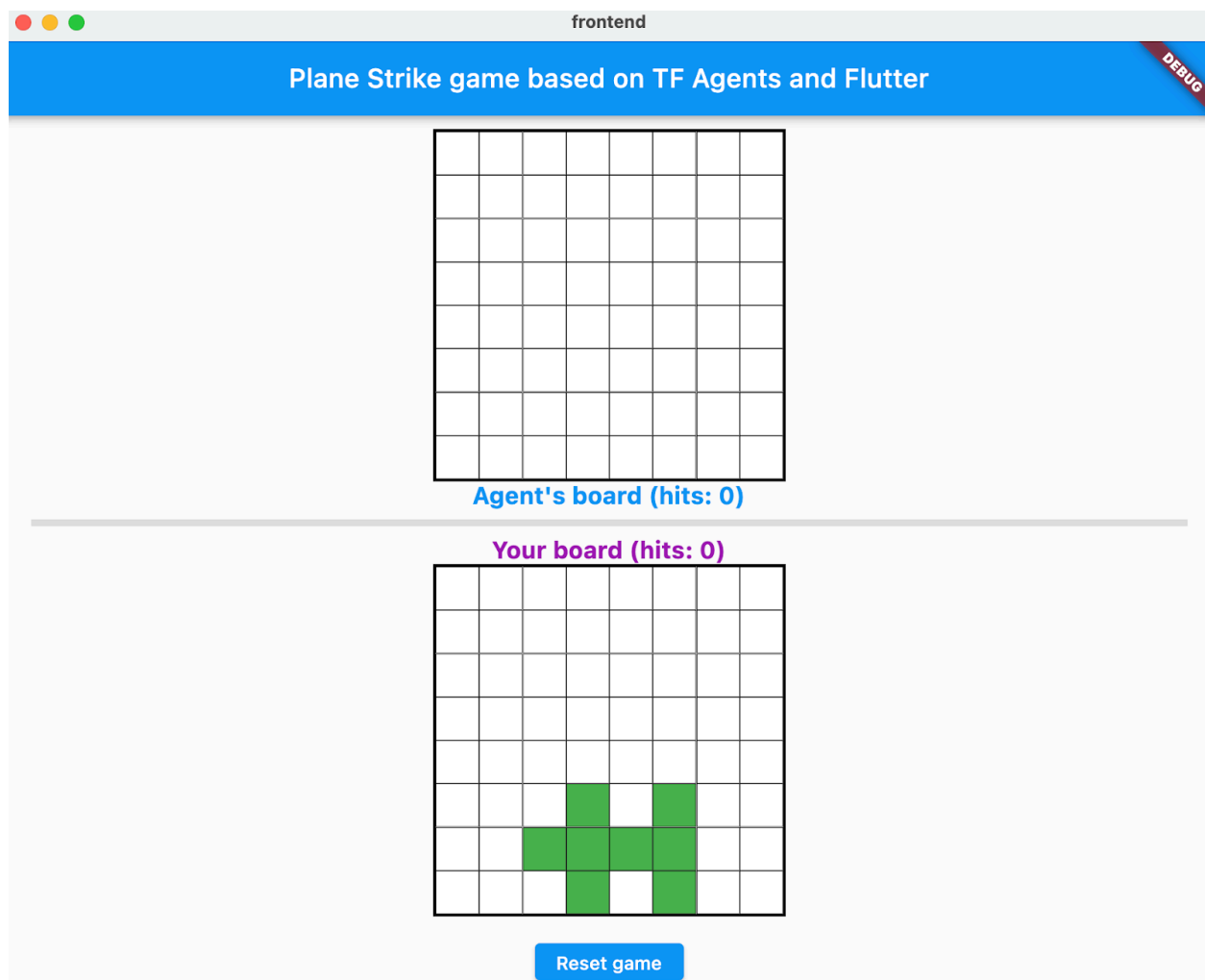
。

3. 按一下「Start debugging」圖示



，然後等待應用程式載入。

4. 按一下代理程式看板中的任一儲存格，即可開始遊戲。



Windows

1. 確認目標裝置在 VSCode 的狀態列中設為

Windows (windows-x64)

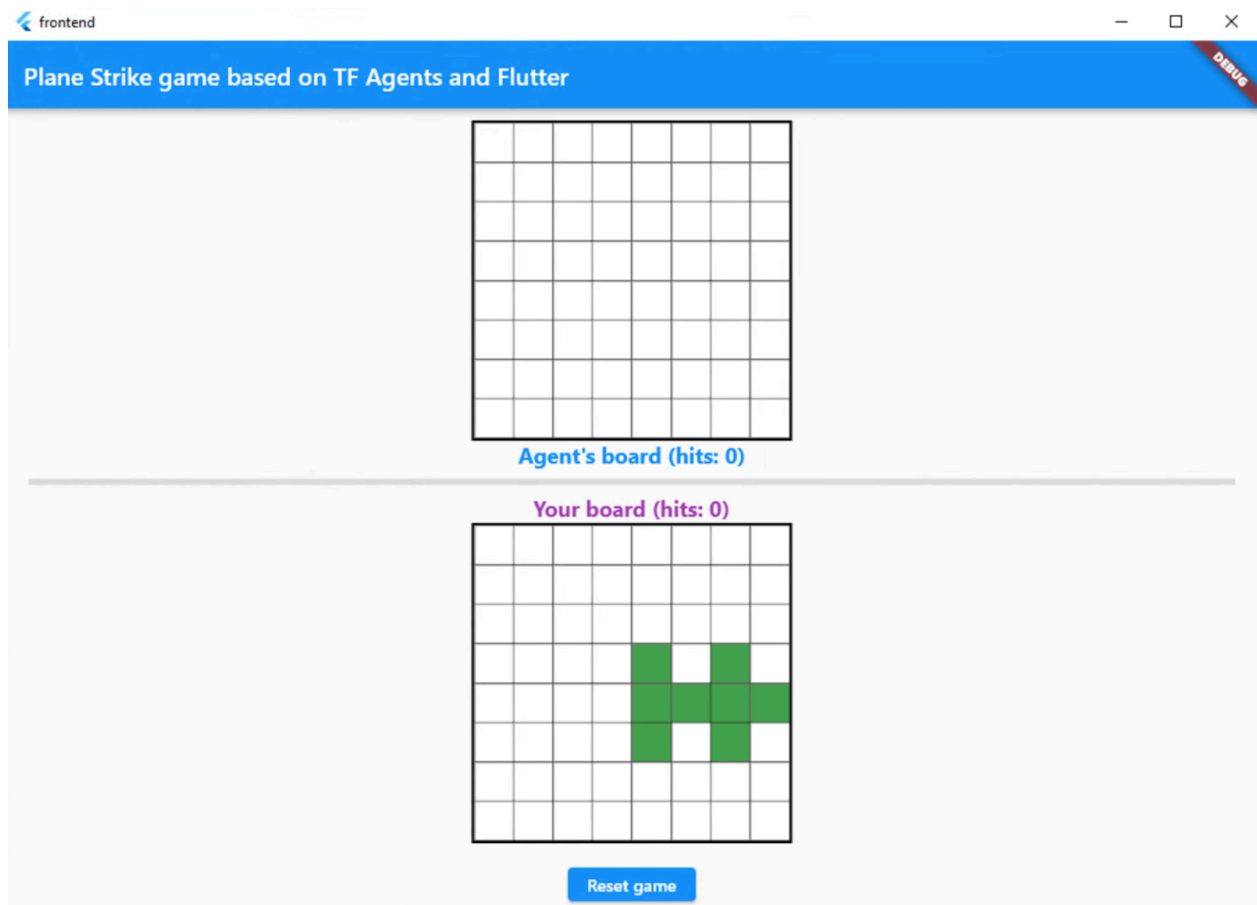
。

2. 按一下「Start debugging」圖示



，然後等待應用程式載入。

3. 按一下代理程式看板中的任一儲存格，即可開始遊戲。



12. 步驟 6：為網頁平台啟用 Flutter 應用程式

您還可以為 Flutter 應用程式新增網頁支援功能。根據預設，Flutter 應用程式會自動啟用網頁平台，因此您只需要啟動即可。

- 1. 確認目標裝置在 VSCode 的狀態列中設為

Chrome (web-javascript)

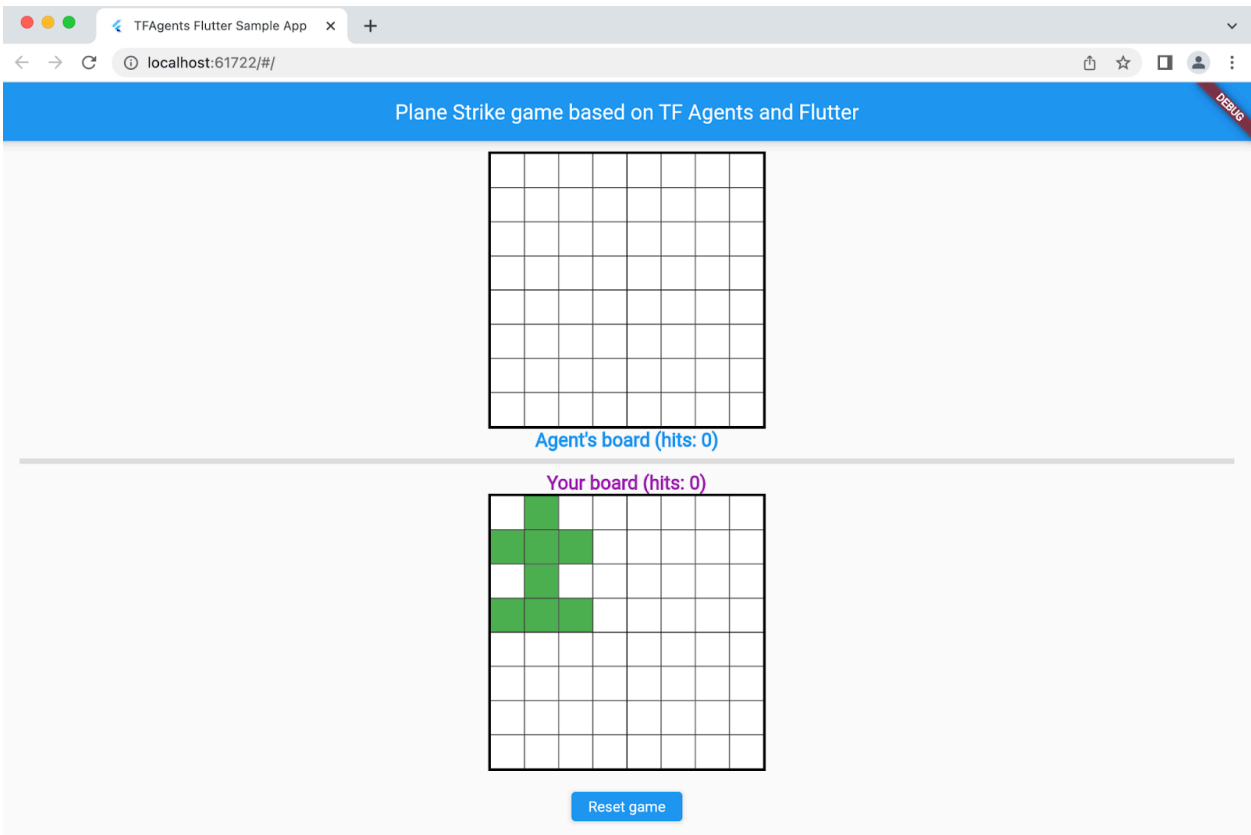
。

- 2. 按一下「開始偵錯」



，然後等待應用程式在 Chrome 瀏覽器中載入。

- 3. 按一下代理程式看板中的任一儲存格，即可開始遊戲。



13. 恭喜

您已建構桌遊應用程式，並透過 ML 輔助代理程式與人類玩家對戰！

注意：Flutter 現在提供[休閒遊戲工具包](#)，內含廣告、音效和音樂、應用程式內購等預先建構的功能。雖然本程式碼研究室不會使用工具包，但如果您想自行建構遊戲，[休閒遊戲工具包](#)是不錯的起點。

瞭解詳情

- [TensorFlow Agents 首頁](#)
 - [TensorFlow Serving 首頁](#)
 - [Flutter 首頁](#)
 - [Flutter 休閒遊戲工具包](#)
 - [TensorFlow 強化學習影片系列](#)
 - [DeepMind 和 UCL 的 2021 年強化學習講座系列](#)
-